LangGraph

Why Do We Need LangGraph?

Let's assume we don't know anything about:

- Storing progress
- Pause & resume
- Checkpointing
- Parallel branches
- Structured execution

We just know how to call an LLM.

Step 1: What People Normally Do

Most beginners build AI systems like this:

```
async function runAgent() { const plan = await planner() const data = await f
etchData(plan) const result = await analyze(data) return result }
```

Or they use a `while` loop for agents.

Everything runs inside one function.

When it finishes → done.

If it crashes → everything is lost.

What Problems Appear?

1️⃣ If system crashes in middle

Suppose:

- Planner finished

- Financial data fetched
- News fetch in progress
- Server crashes

What happens?

👉 You must start from beginning.

👉 You don't know where it stopped.

👉 You manually add logging.

There is no structured progress tracking.

2️⃣ You Can't Pause

Suppose you want:

- Human approval before final answer
- Or wait 2 hours before next step

In normal function flow, you can't easily pause execution.

Because execution lives inside stack memory.

3️⃣ Parallel Becomes Messy

Suppose you want:

- Fetch financial data
- Fetch news data
- In parallel
- Then merge

You must manually write Promise logic and merging logic.

And handle race conditions yourself.

4️⃣ Retry Becomes Dangerous

If one branch fails:

- You must manually retry only that part.
- Ensure state doesn't corrupt.

- Avoid re-running successful branches.

Very messy in raw JS.

Core Problem

In normal code:

Execution is trapped inside a function call.

It is:

- Temporary
- Not checkpointed
- Not resumable
- Not replayable
- Hard to debug

What LangGraph Changes

LangGraph treats execution as:

```
currentStep + state
```

Instead of a function call,

execution becomes a structured object.

Now:

- Every step is isolated
- State is stored
- Progress is known
- You can resume
- You can replay

- You can parallel safely
- You can loop safely
- You can add human-in-loop

🧩 Think Like This

Without LangGraph:

> AI workflow = big function

With LangGraph:

> AI workflow = state machine

Nodes = work

Edges = control

State = memory

Runtime = engine

The Real Reason We Need LangGraph

LLMs are non-deterministic.

Production systems must be deterministic.

LangGraph gives you:

- Deterministic control over non-deterministic intelligence.

That's the real reason.

🟦 Why Do We Need LangGraph?

(Using PDF → VectorDB Example)

1️⃣ The Simple Version (What Everyone Builds First)

Suppose we have a PDF.

Goal:

- Break it into chunks
- Create embeddings
- Store them in Vector DB

Normal implementation:

```
for (let i = 0; i < chunks.length; i++) { const embedding = await embed(chunk
s[i]) await storeInVectorDB(embedding) }
```

This works.

For small projects, this is fine.

2️⃣ Real-World Problems Start Appearing

Now imagine:

- PDF has 1000 chunks
- System crashes at chunk 237

What happens?

❌ You don't know where it stopped

❌ You may duplicate embeddings

❌ You must restart from beginning

❌ You manually add logs

❌ You manually track index

❌ You write custom resume logic

🔥 Important Question

Where is progress stored?

Answer:

Nowhere.

Execution lives inside the loop.

When the function dies → progress dies.

3️⃣ The Core Problem

In normal JS:

> Execution is trapped inside a function call.

It is:

- Temporary
- Not checkpointed
- Not resumable
- Not replayable
- Hard to debug

4️⃣ First Mental Shift

Instead of thinking:

```
for each chunk
```

Think:

```
Execution = currentStep + state
```

State could be:

```
{ currentIndex: 237, totalChunks: 1000 }
```

If this is saved after every chunk,

crash is not scary anymore.

You resume from 237.

---

5 Break Big Loop Into Steps (Nodes)

Instead of one big loop, divide into atomic steps:

1. loadPDF
2. createChunk
3. embedChunk
4. storeEmbedding

Graph structure:

```
createChunk ↓ embedChunk ↓ storeEmbedding ↓ (back to createChunk)
```

Now:

- Each step is isolated
- Each step can be checkpointed
- Each step can be retried safely

Loop exists in structure, not in `while`.

---

6 What LangGraph Actually Gives

LangGraph formalizes this idea.

It gives:

✅ Step Boundaries

Each node is atomic.

✅ State Management

State is structured and persistent.

✅ Automatic Checkpointing

Progress can be stored after every node.

✅ Safe Retry

If `storeEmbedding` fails:

Only that step reruns.

✅ Deterministic Flow

Edges control what runs next.

✅ Parallel Execution

If needed, chunks can embed in parallel safely.

7️⃣ Without LangGraph vs With LangGraph

Normal Loop	LangGraph
Execution hidden in stack	Execution structured
No built-in checkpoint	Checkpoint ready
Manual resume logic	Resume from step
Risk of duplicate work	Step-level retry
Hard to pause	Natural pause
Hard to parallel safely	Deterministic parallel

---

8 The Real Reason We Need LangGraph

LLMs and long workflows are:

- Non-deterministic
- Long-running
- Failure-prone

Production systems must be:

- Deterministic
- Resume-safe
- Replayable
- Structured

LangGraph gives:

> Deterministic control over long-running AI workflows.

We will start doing from here

Simple version:

1. Load PDF
2. Create **all chunks first**
3. Then loop:
    - Embed one chunk
    - Store in Pinecone
    - Move to next

Clean. Deterministic. One-by-one.

🧱 Final Architecture

```
START ↓ loadPDF ↓ createAllChunks ↓ embedChunk ↓ storeEmbedding ↓ checkIfDon
e? ├── yes → END └── no → embedChunk
```

Loop exists between:

`storeEmbedding → embedChunk`

State Design

Since we create all chunks first, state must store them.

```
{ pdfText: null, chunks: null, currentIndex: 0, currentEmbedding: null }
```

That's enough.

Full LangGraph Code (Clean Version)

1 Define State

```
import { Annotation } from "@langchain/langgraph" const State = Annotation.Root({ pdfText: Annotation.string().optional(), chunks: Annotation.any().optional(), currentIndex: Annotation.number().default(() => 0), currentEmbedding: Annotation.any().optional() })
```

Nodes

Load PDF

```
async function loadPDFNode(state) { const text = await loadPDF("file.pdf") return { pdfText: text } }
```

Create All Chunks

```
async function createChunksNode(state) { const chunks = chunkText(state.pdfText) return { chunks } }
```

Embed One Chunk

```
async function embedChunkNode(state) { const chunk = state.chunks[state.currentIndex] const embedding = await embed(chunk) return { currentEmbedding: embedding } }
```

Store in Pinecone

```
async function storeEmbeddingNode(state) { await pinecone.upsert([ { id: Stri
ng(state.currentIndex), values: state.currentEmbedding } ]) return { currentI
ndex: state.currentIndex + 1, currentEmbedding: null } }
```

---

3 Build Graph

```
import { StateGraph } from "@langchain/langgraph" const graph = new StateGrap
h(State) graph.addNode("loadPDF", loadPDFNode) graph.addNode("createChunks",
createChunksNode) graph.addNode("embedChunk", embedChunkNode) graph.addNode
("storeEmbedding", storeEmbeddingNode) graph.setEntryPoint("loadPDF")
```

---

4 Fixed Edges

```
graph.addEdge("loadPDF", "createChunks") graph.addEdge("createChunks", "embed
Chunk") graph.addEdge("embedChunk", "storeEmbedding")
```

---

5 Loop Condition

```
graph.addConditionalEdges( "storeEmbedding", (state) => { if (state.currentIn
dex >= state.chunks.length) return "__end__" return "embedChunk" } )
```

---

6 Compile & Run

```
const app = graph.compile() await app.invoke({})
```

What This Guarantees

- All chunks created first
- Then embeddings happen one-by-one
- Pinecone gets one vector at a time
- Crash at chunk 237?
  - Resume from `currentIndex`
- Loop is structural, not while-loop

Now I'll stop here.

Do you want next:

- Add checkpointer (resume after crash)?
- Or visualize execution step-by-step?

Second Solution

- Take chunks in batches
- Embed batch (parallel inside node)
- Insert batch into Pinecone (bulk)
- Loop until done

Clean. Deterministic. Fast.

🧠 State Design (Batch Version)

```
{ chunks, // already created batchStartIndex: 0, batchSize: 50, currentBatch: null, batchEmbeddings: null }
```

🏗️ Architecture

```
createBatch ↓ embedBatch ↓ storeBatch ↓ checkIfDone? ├── yes → END └── no → createBatch
```

🧩 Nodes

1️⃣ Create Batch

```
async function createBatchNode(state) { const start = state.batchStartIndex const end = start + state.batchSize const batch = state.chunks.slice(start, end) return { currentBatch: batch } }
```

2️⃣ Embed Batch (Parallel Inside Node)

```
async function embedBatchNode(state) { const embeddings = await Promise.all( state.currentBatch.map(chunk => embed(chunk)) ) return { batchEmbeddings: emb eddings } }
```

3️⃣ Store Batch (Bulk Insert)

```
async function storeBatchNode(state) { const vectors = state.batchEmbeddings. map((embedding, i) => ({ id: String(state.batchStartIndex + i), values: embed ding })) await pinecone.upsert(vectors) return { batchStartIndex: state.batch StartIndex + state.batchSize, batchEmbeddings: null, currentBatch: null } }
```

🧠 Graph Definition

```
import { StateGraph } from "@langchain/langgraph" const graph = new StateGrap h(State) graph.addNode("createBatch", createBatchNode) graph.addNode("embedBa tch", embedBatchNode) graph.addNode("storeBatch", storeBatchNode) graph.setEn tryPoint("createBatch") graph.addEdge("createBatch", "embedBatch") graph.addE dge("embedBatch", "storeBatch") graph.addConditionalEdges( "storeBatch", (sta te) => { if (state.batchStartIndex >= state.chunks.length) return "__end__" r eturn "createBatch" } ) const app = graph.compile()
```

🔥 What This Gives You

- Embeddings created in parallel (inside node)

- Pinecone insert in bulk
- Loop handled by graph
- Resume possible (if checkpointer added)
- Deterministic
- Much faster than single-chunk

Third Solution

You want:

- load PDF
- Create chunks in batches (not all at once)
- Send that batch to embedding
- Store batch in Pinecone
- Repeat

No huge memory usage.

No giant chunk array.

Fully streaming in batches.

Perfect.

🧠 Final Optimized Architecture

```
loadPDF ↓ createChunkBatch ↓ embedBatch ↓ storeBatch ↓ checkIfDone? ├── yes → END └── no → createChunkBatch
```

Now chunking is also batched.

🧱 State Design (Streaming + Batch)

```
{ pdfText, textCursor: 0, // where we are in PDF text chunkSize: 1000, overlap: 200, batchSize: 20, // how many chunks per batch currentChunkBatch: null, currentEmbeddings: null }
```

Important:

We do NOT store all chunks.

We generate them batch-by-batch.

🧩 Node 1 — loadPDF

```
async function loadPDFNode(state) { const text = await loadPDF("file.pdf") return { pdfText: text } }
```

🧩 Node 2 — createChunkBatch

Creates multiple chunks starting from `textCursor`.

```
async function createChunkBatchNode(state) { const chunks = [] let cursor = s
tate.textCursor for (let i = 0; i < state.batchSize; i++) { if (cursor >= sta
te.pdfText.length) break const end = cursor + state.chunkSize const chunk = s
tate.pdfText.slice(cursor, end) chunks.push({ id: cursor, content: chunk }) c
ursor += state.chunkSize - state.overlap } return { currentChunkBatch: chunk
s, textCursor: cursor } }
```

Now chunking is incremental.

🧩 Node 3 — embedBatch

```
async function embedBatchNode(state) { const embeddings = await Promise.all(
state.currentChunkBatch.map(c => embed(c.content)) ) return { currentEmbeddin
gs: embeddings } }
```

🧩 Node 4 — storeBatch

```
async function storeBatchNode(state) { const vectors = state.currentEmbedding
s.map((embedding, i) => ({ id: String(state.currentChunkBatch[i].id), values:
embedding })) await pinecone.upsert(vectors) return { currentChunkBatch: nul
l, currentEmbeddings: null } }
```

🧠 Graph Definition

```
const graph = new StateGraph(State) graph.addNode("loadPDF", loadPDFNode) gra
ph.addNode("createChunkBatch", createChunkBatchNode) graph.addNode("embedBatc
h", embedBatchNode) graph.addNode("storeBatch", storeBatchNode) graph.setEntr
yPoint("loadPDF") graph.addEdge("loadPDF", "createChunkBatch") graph.addEdge
("createChunkBatch", "embedBatch") graph.addEdge("embedBatch", "storeBatch")
graph.addConditionalEdges( "storeBatch", (state) => { if (state.textCursor >=
state.pdfText.length) return "__end__" return "createChunkBatch" } ) const ap
p = graph.compile()
```

🔥 Why This Is Best

✅ Memory efficient

✅ No full chunk array

✅ Parallel embedding

✅ Batch DB insert

✅ Resume-safe (if checkpointer added)

✅ Deterministic

✅ Production-grade

Now we do the **real production part** checkpointing.

Right now your graph runs, but if crash happens → restart from beginning.

To fix that, we add a **checkpointer**.

LangGraph supports built-in checkpointing.

🧠 What Checkpointer Does

After every node:

It stores:

```
currentNode + state
```

So if crash happens:

Graph resumes from last completed node.

✅ Option 1 — In-Memory (for learning)

```
import { MemorySaver } from "@langchain/langgraph"
```

```
const checkpointer = new MemorySaver() const app = graph.compile({ checkpoint
er })
```

⚠️ This is NOT durable.

If process dies → memory gone.

Good for demo only.

✅ Option 2 — Persistent (Real Production)

Use SQLite or Postgres.

Install

```
npm install @langchain/langgraph @langchain/langgraph-checkpoint-sqlite
```

SQLite Checkpointer

```
import { SqliteSaver } from "@langchain/langgraph-checkpoint-sqlite" const ch
eckpointer = await SqliteSaver.fromConnString( "file:checkpoints.db" ) const
app = graph.compile({ checkpointer })
```

Now state is stored inside SQLite DB.

🔥 Important: You Must Provide thread_id

Checkpointing works per execution thread.

When invoking:

```
await app.invoke( initialState, { configurable: { thread_id: "pdf-processing-
1" } } )
```

This is critical.

`thread_id` identifies the workflow instance.

🔁 What Happens Now?

After each node:

LangGraph stores:

- node name
- full state

If crash happens:

Call again with same `thread_id`.

LangGraph automatically resumes.

You do NOT manually manage state.

🧠 In Your Optimized Pipeline

If crash happens after:

- storeBatch completed
- textCursor updated

Then resume starts from:

```
createChunkBatch
```

With correct `textCursor`.

No duplication.

No restart.

🔥 Final Production Pattern

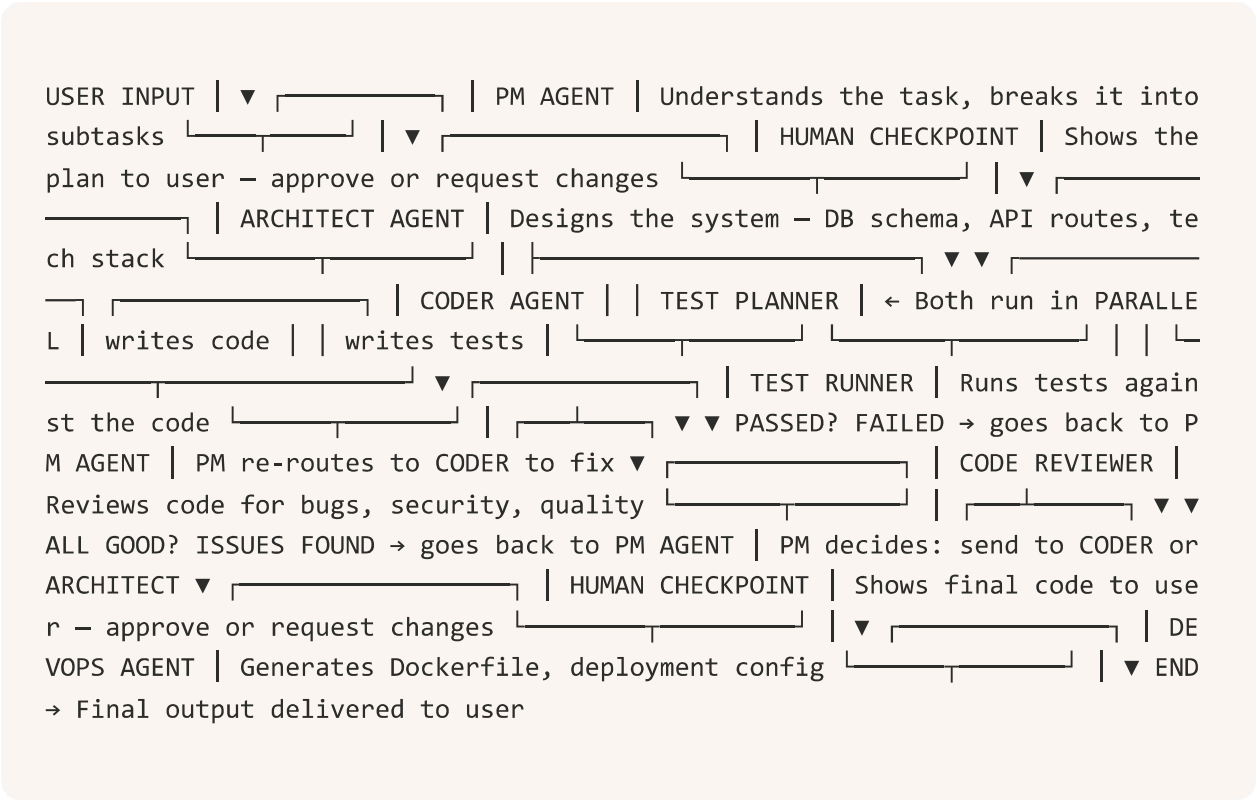
```
const app = graph.compile({ checkpointer }) await app.invoke( {}, { configura
ble: { thread_id: "pdf-001" } } )
```

That's it.

Final Project

AI Software Development Team

User gives a task → **"Build a blog API with auth"**



That's it. That's the entire flow. Input comes in at top, travels through each agent, loops back if something fails, and exits at the bottom.

PM Agent , What Happens When It Receives Input

User says: **"Build a blog API with auth"**

PM Agent is the first node that receives this. Its job is simple — **don't build anything, just THINK and PLAN.**

What PM Agent Does:

Step 1: Reads the user's request

Step 2: Sends it to Gemini with a prompt like — *"You are a project manager. Break this task into clear subtasks. Identify dependencies between tasks. Rate the overall complexity."*

Step 3: Gemini responds with a structured plan:

```
Plan: ├── Task 1: Design database schema (users, posts, comments) ├── Task 2: Design API routes and endpoints ├── Task 3: Set up project with Express + MongoDB ├── Task 4: Build auth system (register, login, JWT) ├── Task 5: Build CRUD for posts ├── Task 6: Build CRUD for comments ├── Task 7: Add middleware (auth, error handling, rate limiting) ├── Task 8: Write tests │ ├── Dependencies: Task 1,2 must finish before 3-7. Task 4 before 5,6. ├── Complexity: HIGH
```

Step 4: PM Agent also decides — **should I ask the human before proceeding?**

The rule is simple:

- Complexity HIGH → yes, ask human to approve the plan first
- Complexity LOW → skip human, directly send to Architect

Step 5: PM Agent passes this plan forward to the next node.

But PM Agent Is NOT Done Forever

This is the important part. PM Agent gets called **again and again** throughout the flow. It's like a manager who keeps checking in:

```
FIRST CALL: User input comes in → PM creates the plan → sends to Human/Archit
ect SECOND CALL: Tests failed → PM reads the failures → re-routes to Coder wi
th fix instructions THIRD CALL: Reviewer found critical design flaw → PM re-r
outes to Architect to redesign FOURTH CALL: Human requested changes → PM upda
tes the plan → re-routes accordingly FIFTH CALL: Everything passed → PM route
s to DevOps → then END
```

Every time PM Agent is called, it looks at **what just happened** and decides **what should happen next**. It's the traffic controller of the entire graph.

Architect Design:

What Architect Agent Does:

Step 1: Reads the plan from PM Agent

Step 2: Sends it to Gemini with a prompt like — *"You are a senior software architect. Based on this plan, design the complete system architecture."*

Step 3: Gemini responds with:

Tech Stack Decision:

```
Runtime: Node.js Framework: Express Database: MongoDB with Mongoose Auth: JWT
with bcrypt Validation: Joi
```

Database Schema:

```
Users → { name, email, password, role, createdAt } Posts → { title, body, aut
hor (ref: Users), tags, createdAt } Comments → { text, author (ref: Users), p
ost (ref: Posts), createdAt }
```

API Routes:

```
POST /auth/register POST /auth/login GET /posts POST /posts GET /posts/:id PU
T /posts/:id DELETE /posts/:id POST /posts/:id/comments GET /posts/:id/commen
ts
```

Folder Structure:

```
src/ ├── index.js ├── config/ ├── routes/ ├── controllers/ ├── models/ ├── mi
ddleware/ └── utils/
```
